

WhatsApp QR Auto-Reply Setup

Meta WhatsApp Cloud API | TurfOn24 booking flow

Goal	When a customer messages your WhatsApp Business number, reply with the turf payment QR as an actual image attachment.
Meta message cost	Free-form image replies are free while Meta's 24-hour customer-service window is open. Template messages outside the window may be charged.
Project status	The repository can upload and send the PNG through Cloud API from the admin action. The inbound webhook that triggers automatic replies is not implemented yet.

Prepared 26 September 2026. Menu labels in Meta may change; use the linked official documentation if a screen differs.

1. How the finished flow works

- 1 The customer starts a WhatsApp conversation by messaging your WhatsApp Business Platform number.
- 2 Meta sends an HTTPS webhook to your app with the customer phone number and inbound message.
- 3 Your server verifies the webhook, finds one active pending booking for that number, and sends **public/logo-assets/qrcodepng.png** as image media with the booking amount and reference.
- 4 The customer pays using a UPI app. Your staff checks the bank/UPI transaction and marks the payment paid in the admin Payments tab.

Meta opens a 24-hour customer-service window when the customer messages. Free-form text and image replies are allowed in that window. If it expires, a free-form image reply is not allowed; an approved template may be required and can cost money.

2. Before you start: protect your personal number

You need a Meta Business Portfolio, a WhatsApp Business Account (WABA), and a phone number registered for Cloud API. The number must be yours and able to receive a verification code by SMS or voice call.

Do not register your everyday personal number until you understand the effect. Meta's direct registration docs say a number already in WhatsApp Messenger must be removed from that account before direct Cloud API registration. Some eligible WhatsApp Business App users can use a coexistence onboarding flow, but availability depends on Meta's eligibility and onboarding path. Test with Meta's test number or use a spare SIM first.

- Meta Developer account and developer registration.
- Meta Business Portfolio and WhatsApp Business Account.
- A test phone number for initial development; later, a business phone number you control.
- A public HTTPS webhook URL reachable without a Vercel login page.

3. Create the Meta app and test number

- 1 Open [Meta App Dashboard](#) and register as a developer if prompted.
- 2 Create an app with the WhatsApp use case, usually labeled **Connect with customers through WhatsApp**. Select an existing business portfolio or create one.
- 3 Open the app's WhatsApp Quickstart / API Setup page and connect or create a WhatsApp Business Account.
- 4 Use the generated test number first. Generate a temporary access token, select the test sender, and add your own phone as a permitted test recipient.
- 5 Send a test message and reply to it from your phone. This proves the API setup and opens a test customer-service window.
- 6 Record the **Phone Number ID** and **WhatsApp Business Account ID** shown in API Setup. They are IDs, not phone numbers.

The temporary token is for testing only and expires. Do not put it in frontend code or share it in chat.

4. Add a production business number

- 1 In WhatsApp API Setup, add a business phone number and complete Meta's SMS or voice verification.
- 2 Configure the business display name and profile details. Complete any identity or business verification steps Meta requests for your account.
- 3 Finish registering the phone number for Cloud API and confirm its status is **Connected** in WhatsApp Manager.
- 4 Keep the test number until production sending and webhook replies are verified.

The business number receives customer messages. Customers can continue to use their ordinary WhatsApp accounts; they do not need a business account.

5. Create a long-lived server token

- 1 Open Meta Business Settings and go to **Users > System Users**. Create a system user for the integration.
- 2 Assign the system user access to the Meta app and the WhatsApp Business Account / phone-number assets.
- 3 Generate a system-user access token for the app. Select the WhatsApp permissions Meta lists for your app, including **whatsapp_business_messaging** and **whatsapp_business_management**. Add **business_management** only if the setup flow requires it.
- 4 Copy the token once and store it in a password manager until you enter it directly into Vercel. Never commit it, add a VITE_ prefix, or paste it into chat.

6. Webhook setup for automatic replies

Automatic replies need a webhook endpoint in this codebase. It must support both Meta's verification request and incoming message POSTs. The current repository does not yet contain this endpoint, so do not enter a callback URL in Meta until the webhook code has been added and deployed.

Webhook endpoint requirements

- **GET verification:** check Meta's `hub.mode`, `hub.verify_token`, and `hub.challenge` query parameters; return the challenge only when the configured verify token matches.
- **POST verification:** verify X-Hub-Signature-256 using the raw request body and your Meta App Secret. Reject invalid signatures.
- **Message handling:** subscribe to the **messages** field, read the sender ID, normalize it to international digits, and record the inbound timestamp.
- **Safe booking match:** find exactly one active `PAYMENT_PENDING` session for the sender. If there are zero or multiple matches, do not send the QR automatically; ask staff to resolve the booking.
- **Duplicate protection:** store inbound Meta message IDs and ignore retries already processed. Return HTTP 200 promptly after safely recording the event.
- **Reply:** upload the PNG to Meta media, then send an image message using the returned media ID. The sender code already performs these two API calls when triggered.

Meta retries webhook deliveries when your callback fails. Idempotency is essential so retries do not send duplicate QR images.

7. Configure the Meta webhook dashboard

- 1 Deploy the webhook route to a public HTTPS URL, for example **https://your-public-domain/api/whatsapp-webhook**. The endpoint cannot be behind Vercel Preview Protection because Meta cannot complete a browser login.
- 2 In the Meta App Dashboard, open WhatsApp Configuration (or the WhatsApp use-case configuration). Enter the callback URL and the same verify token you configured on the server.
- 3 Complete Meta's webhook verification challenge. Then subscribe the app to the WhatsApp Business Account and the **messages** webhook field.
- 4 Send a test message from your phone to the business number and verify that the webhook receives it. Do not test using a customer booking until the test is reliable.

8. Vercel Preview environment variables

In Vercel, open the project Settings > Environment Variables. Add the following for **Preview** only while testing, then redeploy the preview deployment.

Name	Where it comes from / purpose
WHATSAPP_ACCESS_TOKEN	System-user access token with WhatsApp messaging permission.
WHATSAPP_PHONE_NUMBER_ID	Phone Number ID shown in Meta WhatsApp API Setup.
WHATSAPP_GRAPH_API_VERSION	Optional. Current code default is v25.0; use a version supported by your Meta app.
WHATSAPP_APP_SECRET	Meta app secret, used only to validate incoming webhook signatures. This webhook integration still needs implementation.
WHATSAPP_WEBHOOK_VERIFY_TOKEN	A random value you create; enter the same value in Meta webhook configuration.

The first three variables are already used by the QR image sender. The last two are names recommended for the future webhook implementation; adding them alone will not create the webhook.

9. Verification checklist

- Use the Meta test number and an allow-listed test recipient first.
- Send a customer message and confirm a webhook event arrives exactly once.
- Confirm the backend matches only the correct active booking session for that customer number.
- Verify the received WhatsApp message contains an image attachment, not a public Vercel link.
- Test an expired 24-hour window; the system should not attempt a free-form send.
- Test duplicate webhook delivery; the customer should receive only one QR.
- Keep the access token and App Secret server-side, and set live values only after the test works.

10. Cost and customer-service window

Meta's current pricing says non-template messages, including image replies, are free during an open customer-service window. The window lasts 24 hours after the customer messages the business. Outside

the window, Meta requires an approved template and may charge per delivered message. Hosting or Vercel usage is separate from Meta message fees.

[Meta WhatsApp pricing](#)

11. What is already in this repository

- The authenticated admin image sender is implemented in `server/admin/whatsapp-notify.js`.
- Meta media upload and image-message calls are in `server/_lib/whatsapp-delivery.js`.
- The QR asset is `public/logo-assets/qrcodepng.png` and is included in the Vercel function bundle.
- Incoming webhook route, inbound-message matching, and automatic webhook-triggered sending are not implemented yet.
- Checkout currently does not automatically send WhatsApp messages. The website uses its existing UPI QR/payment flow; staff can mark a payment paid from the admin Payments tab after checking the bank transaction.

The immediate next code task is to add and test the signed Meta webhook route and the pending-booking match. Once that is deployed, configure the callback URL in Meta and set the webhook secrets in Vercel Preview.

12. Official setup references

- [Cloud API getting started](#)
- [Cloud API webhooks](#)
- [Business phone numbers](#)
- [Meta WhatsApp pricing](#)
- [Vercel environment variables](#)